Desenvolvimento de uma Aplicação Web para Consulta de Dados de Servidores do Executivo Federal

Pedro Machado[1], **Marcos Freitas**[1], **Alexsander Santos**[1]

[1]Universidade Federal do Rio Grande (FURG)
Sistemas para Internet II – Rio Grande – RS – Brasil

pedro.machado@furg.br, mcopellofreitas@hotmail.com

alexsander.oml@gmail.com

Abstract. *This report describes the development of a full stack application for querying public data about active and retired employees of the Brazilian Federal Executive Branch. The solution combines PostgreSQL, a Flask API served by Gunicorn, and a React interface coordinated by Docker Compose. It records daily task allocation, each contributor's Git history, the burndown of two sprints, implemented requirements, executed tests, team analysis, and verified limitations. The tracked sprint backlog totalled 115 Story Points; completed tasks are distinguished from assignment requirements that remain only partially evidenced.*

Resumo. *Este relatório descreve o desenvolvimento de uma aplicação full stack para consulta de dados públicos de servidores ativos e aposentados do Poder Executivo Federal. A solução combina PostgreSQL, API Flask executada pelo Gunicorn e interface React coordenada pelo Docker Compose. São registrados a divisão diária das tarefas, o histórico Git de cada integrante, o burndown de duas sprints, os requisitos implementados, os testes executados, a análise da equipe e as limitações comprovadas. O backlog acompanhado totalizou 115 Story Points; as tarefas concluídas são diferenciadas dos requisitos do enunciado que permanecem apenas parcialmente comprovados.*

1. Introdução e objetivo

O trabalho propôs uma aplicação full stack para consultar informações de servidores ativos e aposentados do Poder Executivo Federal. O enunciado exigiu o uso de GitHub, metodologia ágil, Docker, PostgreSQL, Python, Flask, Gunicorn e uma interface web, além do registro das tarefas diárias, dos commits, do burndown e da análise crítica da equipe [Sistemas para Internet II 2026].

As fontes definidas foram os conjuntos Gestão de Pessoas do Executivo Federal – Aposentados e Gestão de Pessoas do Executivo Federal – Carreiras/Cargos [Portal Brasileiro de Dados Abertos 2026a, Portal Brasileiro de Dados Abertos 2026b]. O escopo obrigatório incluiu consultas por nome exato, similaridade, cargo, UF, órgão e combinações, com apresentação de um ou vários resultados. O aplicativo desktop foi mantido fora do escopo porque corresponde ao item adicional da avaliação.

Este relatório está organizado segundo os itens avaliados: organização da equipe e sprints; tarefas e histórico diário do GitHub; burndown; arquitetura e implementação;

testes; limitações; pontos fortes e fracos; dificuldades e impedimentos; e preparação da entrega.

2. Organização da equipe e metodologia ágil

2.1. Responsabilidades

A equipe foi composta por três integrantes. Pedro Machado exerceu a liderança técnica e concentrou infraestrutura, garantia de qualidade e documentação de gestão. Marcos Freitas atuou no banco de dados e no backend. Alexsander Santos ficou responsável pelo frontend. A Tabela 1 relaciona os nomes usados no relatório às identidades públicas do histórico Git.

Tabela 1. Divisão de responsabilidades da equipe.

Integrante	Usuário no GitHub	Responsabilidades principais
Pedro Machado	`pedromchd`	Liderança técnica, infraestrutura, QA e documentação de gestão.
Marcos Freitas	`marcoscfreitas`	Banco de dados, preparação dos dados e backend.
Alexsander Santos	`alexsander-oml`	Frontend, integração com a API e apresentação dos resultados.

2.2. Sprints e fluxo de trabalho

O desenvolvimento foi dividido em duas sprints de 15 dias. A Sprint 1 ocorreu de 12/09 a 26/09/2026 e a Sprint 2 de 27/09 a 11/10/2026, com 12/10 reservado para o fechamento anterior à entrega. O Jira registrou descrição, responsável, Story Points e estado de cada atividade. As tarefas receberam 1 ou 2 pontos e foram ordenadas pelas dependências entre infraestrutura, dados, backend, frontend, integração, testes e documentação.

As mudanças foram produzidas em branches identificadas pela área e pela chave do Jira. Cada commit foi vinculado a uma única tarefa e preservou a autoria do responsável. Até duas tarefas relacionadas, do mesmo integrante e da mesma área, puderam compartilhar uma branch, sem mistura no mesmo commit. As integrações foram realizadas em `dev`.

Tabela 2. Distribuição final de Story Points por integrante.

Integrante	Sprint 1	Sprint 2	Total
Pedro Machado	16	23	39
Marcos Freitas	22	17	39
Alexsander Santos	18	19	37
Total	56	59	115

O backlog da Sprint 1 totalizou 56 pontos. O da Sprint 2 começou com 57 pontos e recebeu 2 pontos em 09/10 para a revisão final do relatório, totalizando 59; as atividades

registradas foram concluídas nessa data. Esse fechamento em Story Points não substitui a verificação dos requisitos, apresentada na Tabela 6. Dias sem commit foram mantidos no histórico, sem atribuição retroativa de trabalho.

3. Tarefas diárias e histórico do GitHub

As Tabelas 3 e 4 reúnem, por dia e por usuário, as tarefas delegadas e os commits correspondentes. Assim, os IDs do backlog não aparecem isolados: cada registro informa também o resultado produzido. O histórico foi conferido em `dev` até `033a83b`, primeira consolidação da PROJ-72. A presente correção continua vinculada à mesma tarefa; seu próprio hash não pode constar no conteúdo que o determina.

Tabela 3. Tarefas e commits diários da Sprint 1.

Data	Integrante	Tarefas delegadas e commits
12/09	Pedro Machado	`INF-01/PROJ-3`: configurar e ajustar a estrutura inicial do repositório – `82574ee`, `0a50fb5`.
12/09	Marcos Freitas e Alexsander Santos	Sem entrega integrada.
13/09	Pedro Machado	`INF-02/PROJ-4`: organizar backlog e quadro do Jira – atividade de gestão sem commit.
13/09	Marcos Freitas	`BD-01/PROJ-15`: adicionar `psycopg[binary]` às dependências do backend para conexão com PostgreSQL – `1148065`; `BD-02/PROJ-16`: analisar os datasets – `882f94d`.
13/09	Alexsander Santos	`FRONT-01/PROJ-29`: configurar a estrutura do frontend – `58ffbc9`; `FRONT-02/PROJ-30`: definir a identidade visual – `776d42f`.
14/09	Pedro Machado	`INF-03/PROJ-5`: criar a composição inicial – `c8395e6`.
14/09	Marcos Freitas	`BD-03/PROJ-17`: mapear os campos necessários – `d64f588`.
14/09	Alexsander Santos	`FRONT-03/PROJ-31`: criar o layout base – `9ab57fe`.
15/09	Pedro Machado	`INF-04/PROJ-6`: criar a imagem do banco – `9785ee7`.
15/09	Marcos Freitas	`BD-04/PROJ-18`: modelar o schema – `fe3b0d8`.
15/09	Alexsander Santos	`FRONT-04/PROJ-32`: criar o formulário de busca – `ea59923`.
16/09	Pedro Machado	`INF-05/PROJ-7`: criar a imagem do backend – `e4be7c3`.
16/09	Marcos Freitas	`BD-05/PROJ-19`: criar as tabelas do banco – `286d627`.
16/09	Alexsander Santos	`FRONT-05/PROJ-33`: estilizar o formulário – `9febd91`.
17/09	Equipe	Sem entrega integrada.
18/09	Pedro Machado	`INF-06/PROJ-8`: criar a imagem do frontend – `580e9ef`; `INF-07/PROJ-9`: validar a orquestração – `3baff68`.
18/09	Marcos Freitas	`BD-06/PROJ-20`: preparar os dados para importação – `70c9196`.

Data	Integrante	Tarefas delegadas e commits
18/09	Alexsander Santos	FRONT-06/PROJ-34: criar a listagem fictícia – 4123f17; FRONT-07/PROJ-35: estilizar a listagem – f6a31fd.
19/09	Equipe	Sem entrega integrada.
20/09	Pedro Machado	INF-08/PROJ-10: configurar variáveis de ambiente – 493c300; QA-01/PROJ-11: definir o plano de testes – edf7e9d.
20/09	Marcos Freitas	BD-07/PROJ-21: importar dados tratados – d27e820; BD-08/PROJ-22: criar índices – 66f5f0b; BACK-01/PROJ-23: criar o esqueleto Flask – e4513c3.
20/09	Alexsander Santos	FRONT-08/PROJ-36: criar a paginação fictícia – 8570123; FRONT-09/PROJ-37: criar estados simulados – ad846df.
21–22/09	Equipe	Sem entrega integrada.
23/09	Pedro Machado	QA-02/PROJ-12: definir casos de SQL injection – b12a706; LID-01/PROJ-13: registrar o acompanhamento – 79ef587; LID-02/PROJ-14: estruturar o relatório – 26cd934.
23/09	Marcos Freitas	BACK-02/PROJ-24: configurar Gunicorn multithread – ccdf5c4; BACK-03/PROJ-25: criar conexão com PostgreSQL – 4cfa188; BACK-04/PROJ-26: implementar busca exata – 70f63a5.
23/09	Alexsander Santos	FRONT-10/PROJ-38: ajustar a responsividade – 44d97f8.
24–25/09	Equipe	Sem entrega integrada.
26/09	Pedro Machado	LID-01/PROJ-13: encerrar o acompanhamento da Sprint 1 – 5236647.
26/09	Marcos Freitas	BACK-05/PROJ-27: implementar filtros por cargo e UF – 8e440b9; BACK-06/PROJ-28: implementar filtro por órgão – 47e3872.
26/09	Alexsander Santos	Sem entrega integrada.

Tabela 4. Tarefas e commits diários da Sprint 2.

Data	Integrante	Tarefas delegadas e commits
27/09	Pedro Machado	INF-09/PROJ-39: integrar a aplicação ao Compose – 467b7a3.
27/09	Marcos Freitas	BACK-07/PROJ-51: implementar busca combinada – f8a1064.
27/09	Alexsander Santos	FRONT-11/PROJ-61: integrar o frontend à API – cc9f180; FRONT-12/PROJ-62: implementar busca exata – 3703715.

Data	Integrante	Tarefas delegadas e commits
28–29/09	Equipe	Sem entrega integrada.
30/09	Pedro Machado	INF-10/PROJ-40: configurar healthchecks e reinício – fcc9fb3; INF-11/PROJ-41: limitar buscas simultâneas – 555a6a3; INF-12/PROJ-42: aplicar timeout – dd80b71.
30/09	Marcos Freitas	BACK-08/PROJ-52: implementar similaridade – ed9e38f; BACK-09/PROJ-53: implementar paginação – 389c272; BACK-10/PROJ-54: validar parâmetros – 728568f.
30/09	Alexsander Santos	FRONT-13/PROJ-63: implementar similaridade – ef2df83; FRONT-14/PROJ-64: implementar filtros individuais – 4b13c89; FRONT-15/PROJ-65: implementar busca combinada – fb65451.
01–03/10	Equipe	Sem entrega integrada.
04/10	Pedro Machado	QA-03/PROJ-43: executar carga – 1274a0d; QA-04/PROJ-44: registrar resultados e limitações – 569da1f; QA-05/PROJ-45: executar cenários de erro – 98f891d; QA-06/PROJ-46: validar SQL injection – 49fe3e6.
04/10	Marcos Freitas	BACK-11/PROJ-55: tratar erros – 545c692; BACK-12/PROJ-56: padronizar JSON – e1d822f; BD-09/PROJ-57: otimizar similaridade – 1404a59; BACK-13/PROJ-58: adicionar testes – 41eb8da.
04/10	Alexsander Santos	FRONT-16/PROJ-66: integrar paginação – a9ed8db; FRONT-17/PROJ-67: diferenciar resultado único – 09b01a6; FRONT-18/PROJ-68: adicionar feedback – 982d69a; FRONT-19/PROJ-69: ajustar responsividade – 70e40cd.
05/10	Equipe	Sem entrega integrada.
06/10	Pedro Machado	LID-03/PROJ-47: consolidar tarefas e histórico – 3acd6e5; LID-04/PROJ-48: criar o burndown – f2faee7.
06/10	Marcos Freitas	BACK-14/PROJ-59: documentar a API – 24a866c; BACK-15/PROJ-60: corrigir e revalidar respostas – 0a28f98.
06/10	Alexsander Santos	FRONT-20/PROJ-70: corrigir estados de busca e respostas inválidas – 6ab7768.
07–08/10	Equipe	Sem entrega integrada; Marcos Freitas e Alexsander Santos já não possuíam pendências obrigatórias.
09/10	Pedro Machado	LID-05/PROJ-49: registrar a análise crítica – cc1ebfa; LID-06/PROJ-50: permitir a inicialização sem arquivo de ambiente e validar a aplicação em clone limpo – e5647e8; LID-07/PROJ-72: consolidar o relatório – 033a83b.

Data	Integrante	Tarefas delegadas e commits
09/10	Marcos Freitas e Alexsander Santos	Sem pendências obrigatórias.
10/10	Pedro Machado	`LID-07/PROJ-72`: revisar o relatório com o pacote oficial da SBC e alinhá-lo ao enunciado – presente revisão.
10/10	Marcos Freitas e Alexsander Santos	Sem pendências obrigatórias.
11/10	Equipe	Período reservado para fechamento.

3.1. Burndown

O saldo real representa o fim de cada dia. A Sprint 2 mantém a linha-base de 57 pontos e registra a alteração de escopo de 09/10 separadamente: havia 3 pontos restantes, a PROJ-72 acrescentou 2 e as três tarefas do dia concluíram 5 pontos, levando o saldo a zero.

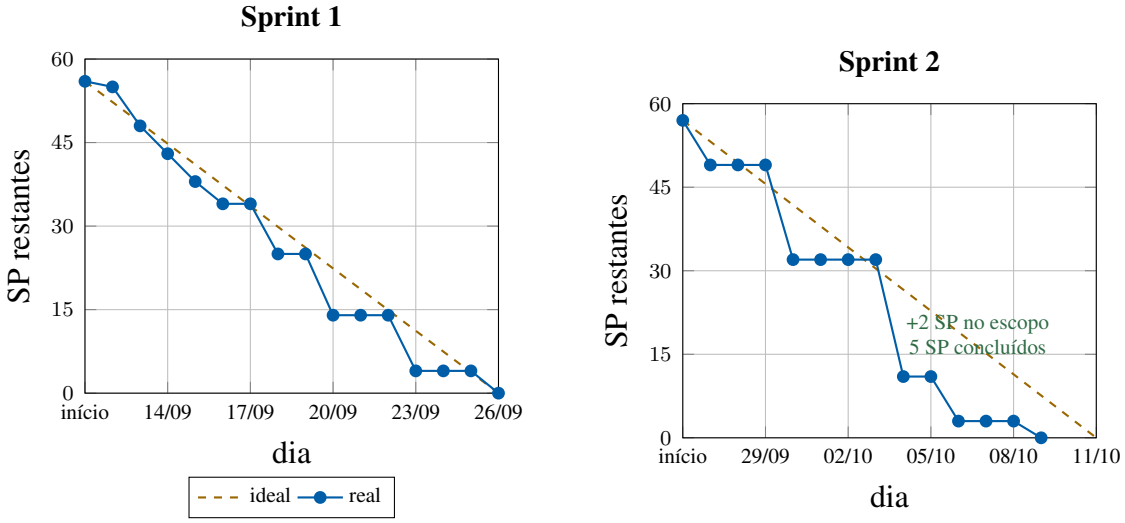


Figura 1. Story Points restantes nas duas sprints. A Sprint 2 terminou com escopo final de 59 SP.

Tabela 5. Fechamento das sprints.

Sprint	Base	Ajuste	Escopo final	Concluídos	Fechamento
Sprint 1	56	0	56	56	26/09
Sprint 2	57	+2	59	59	09/10

4. Arquitetura e implementação

4.1. Docker e execução

A aplicação possui três serviços orquestrados pelo Docker Compose:

- `db`: PostgreSQL 18 Alpine, responsável pelo schema, índices e extensão `pg_trgm`;

- `backend`: Python 3.14, Flask 3.1.3, Gunicorn 26.2.0 e psycopg 3.3.5;
- `frontend`: React 19, TypeScript, Vite 8 e Tailwind CSS 4.

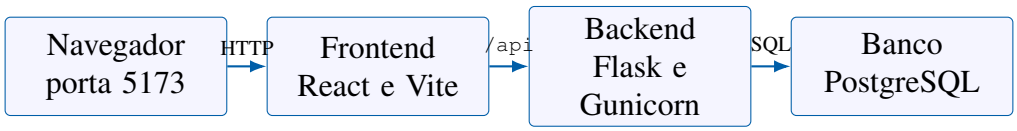


Figura 2. Comunicação entre os módulos da aplicação.

O navegador acessa o frontend pela porta 5173. As requisições iniciadas em `/api` são encaminhadas pelo proxy do Vite ao backend na porta interna 8000. O backend acessa o PostgreSQL pelo nome de serviço `db`; o banco não publica porta para o host. O Gunicorn usa um processo e quatro threads. O Compose limita a API a duas buscas simultâneas e aplica timeout de 2.000 ms. Banco e backend possuem healthchecks e os três serviços usam `restart: unless-stopped`.

A execução a partir da raiz usa o comando recomendado pela documentação do Docker Compose [Docker 2026]:

```
docker compose up --build
```

O comando constrói e inicia os três serviços. Entretanto, um clone limpo cria o schema sem registros públicos. Portanto, a infraestrutura fica disponível, mas consultas úteis dependem da execução prévia do tratamento e da importação. Essa diferença em relação à expressão "pronta para uso" do enunciado é uma limitação da entrega, não um resultado considerado aprovado.

4.2. Tratamento dos dados e PostgreSQL

Foram analisados os conjuntos de Aposentados e Carreiras/Cargos e seus dicionários de dados [Ministério da Gestão e da Inovação em Serviços Públicos 2018, Ministério da Gestão e da Inovação em Serviços Públicos 2019]. A inspeção abrangeu as competências de setembro de 2025 a agosto de 2026. Os recursos de agosto de 2026 estavam indisponíveis e o arquivo de Aposentados de dezembro de 2025 estava vazio.

Os campos utilizados foram nome, cargo, órgão e UF. CPF, matrícula, remuneração e os demais dados pessoais ou funcionais não foram incorporados. A UF não existe no conjunto de Aposentados e permanece nula nesses registros. Os arquivos usam Latin-1 e separação por ponto e vírgula.

O tratamento remove bytes NUL e espaços excedentes; ignora linhas vazias, cabeçalhos e linhas de identificação; trata o delimitador final adicional de Carreiras/Cargos; valida quantidade de colunas e campos obrigatórios; normaliza a UF; rejeita UFs malformadas; remove registros idênticos no mesmo arquivo; e gera CSV UTF-8 com origem, arquivo e linha do registro.

O modelo relacional contém `servidor`, `cargo`, `orgao` e `uf`. A importação usa tabela temporária, insere os domínios distintos e cria os relacionamentos em `servidor`. O processo exige tabelas de destino vazias e interrompe a importação se

as quantidades divergirem. Não existe evidência persistida da contagem final de uma importação pública; por isso, este relatório não atribui um total importado.

A extensão `pg_trgm` sustenta a similaridade. O nome do servidor usa índice GiST, compatível com ordenação por distância textual. Cargo e órgão possuem índices trigram GIN, e as chaves estrangeiras usam índices B-tree.

4.3. Backend e API

A API disponibiliza `GET /health` e `GET /api/servidores`. A consulta aceita filtros exatos por nome, cargo, UF e órgão, similaridade por nome e combinações entre filtros. No parâmetro `similar`, os marcadores `%` dos formatos exemplificados no enunciado são removidos antes da busca. O limite de similaridade varia de 1 a 100, com padrão 100. A paginação usa `page` e `page_size`, com até 100 registros por página.

As respostas usam um envelope JSON com `sucesso`, `mensagem` e `dados`; respostas paginadas incluem `paginacao`. A API diferencia registro único, múltiplos registros, ausência de resultados e página inexistente. UF, números e parâmetros desconhecidos são validados antes do acesso ao banco.

As consultas separam parâmetros do SQL e entradas compatíveis com injeção são recusadas. A API trata entrada inválida, recurso inexistente, método não permitido, excesso de concorrência, falha interna e timeout. A terceira busca simultânea recebe HTTP 429 e o cancelamento pelo PostgreSQL é convertido em HTTP 504.

4.4. Frontend

O frontend apresenta nome, modo exato ou similar, limite da similaridade, cargo, UF e órgão. Os filtros podem ser usados isoladamente ou em conjunto. A interface não consulta ao abrir e impede envio sem filtro.

Cada consulta usa páginas de dez registros. A navegação oferece anterior, próxima e até cinco números de página. A interface implementa estados para resultado único, lista, carregamento, ausência de resultados e erro. Respostas inválidas e falhas de rede produzem mensagens controladas. O layout é responsivo, usa rótulos, regiões de status e alertas e toma o Portal Brasileiro de Dados Abertos como referência visual.

O código desses estados está implementado e o build foi validado. A evidência integrada preservada, contudo, cobre apenas o caminho de uma busca sem resultados em banco vazio. Não há evidência persistida de uma bateria manual do frontend com dados positivos, resultado único, múltiplos resultados e paginação; essa lacuna é registrada na Seção 6.1.

5. Atendimento aos requisitos do enunciado

A Tabela 6 separa implementação, evidência e limitação. O estado “parcial” não invalida o código existente; indica que a evidência preservada não comprova integralmente o comportamento exigido.

Tabela 6. Relação entre requisitos e evidências.

Requisito	Implementação ou evidência	Situação
GitHub, equipe e metodologia ágil	Três identidades, tasks Jira, commits atômicos, duas sprints e histórico diário.	Atendido
Três containers e Dockerfiles	Banco, backend e frontend construídos e iniciados pelo Compose.	Atendido
Aplicação pronta após `up --build`	Serviços iniciam, mas o banco público exige tratamento e importação posteriores.	Parcial
PostgreSQL, preparação e índices	Schema, pipeline, validações e índices existem; falta contagem persistida da importação pública final.	Parcial
Python, Flask e Gunicorn	API executada pelo Gunicorn e validada com PostgreSQL real.	Atendido
Filtros, combinações, JSON, paginação e erros	Implementação e baterias automatizada, funcional, de timeout, concorrência e segurança.	Atendido
Frontend apresentando os dados	Build e caminho integrado sem resultados foram verificados; cenários positivos não possuem evidência persistida.	Parcial
Sobrecarga e proteção	Limite de concorrência, HTTP 429, timeout e disponibilidade posterior foram exercitados.	Atendido
Tarefas, commits, burndown e análise da equipe	Tabelas diárias, Figura 1 e Seção 7.	Atendido
Aplicativo desktop	Item adicional não desenvolvido.	Fora do escopo obrigatório

6. Estratégia e resultados dos testes

A bateria funcional cobriu filtro vazio, espaços, tentativa representativa de injeção, nome inexistente, múltiplos resultados paginados, cargo, UF e órgão inválidos, rota e página inexistentes, parâmetro desconhecido, limites inválidos, combinação inválida, timeout, concorrência e disponibilidade posterior. Na bateria de segurança, os payloads foram enviados aos cinco filtros; as tabelas, contagens e fingerprints permaneceram inalteradas.

No cenário de capacidade, 20 respostas HTTP 200 foram concluídas em 2,921 s, com 6,85 requisições por segundo, mediana de 276,08 ms e p95 de 367,76 ms. Na sobrecarga, o limite recusou 198 requisições com HTTP 429 e permitiu duas respostas HTTP 200. Não houve erro de transporte e a saúde permaneceu disponível.

6.1. Limitações

As limitações comprovadas são:

- o clone limpo inicia os serviços e cria o schema, mas não carrega os arquivos públicos automaticamente;

Tabela 7. Consolidação das verificações executadas.

Verificação	Resultado
Backend automatizado	12 de 12 testes aprovados.
API funcional e erros	20 de 20 cenários aprovados contra Flask, Gunicorn e PostgreSQL reais.
SQL injection	45 de 45 casos aprovados, sem alteração de tabelas, contagens ou fingerprints.
Carga de capacidade	20 requisições, concorrência 2, todas HTTP 200 e sem erro de transporte.
Sobrecarga	200 requisições, concorrência 32; 2 HTTP 200 e 198 HTTP 429, sem erro de transporte.
Disponibilidade posterior	`/health` respondeu HTTP 200 após a sobrecarga.
Timeout real	Consulta bloqueada retornou HTTP 504 em aproximadamente 2 segundos.
Clone limpo	Três imagens construídas; banco e backend saudáveis; frontend e `/health` com HTTP 200.
Integração do frontend	Busca percorreu frontend, proxy e API e apresentou o estado sem resultados.

- não existe contagem persistida da importação pública final;
- a evidência integrada do frontend cobre banco vazio, não os cenários positivos completos;
- a execução final em clone limpo de 09/10 está resumida neste relatório, mas não possui log independente versionado; `docs/validacao-compose.md` registra somente a validação preliminar de 18/09;
- as baterias completas funcional, de injeção e de carga foram executadas em 04/10, antes da correção do backend de 06/10; depois dela, os 12 testes automatizados passaram em 09/10, mas as baterias completas não foram repetidas;
- os testes ocorreram localmente, com uma instância do backend e, nos testes de carga, 100.000 registros sintéticos;
- não houve teste prolongado de estabilidade nem distribuição entre múltiplas instâncias;
- a capacidade configurada é de duas buscas simultâneas e rejeita o excedente com HTTP 429;
- o aplicativo desktop adicional não foi desenvolvido.

7. Análise crítica da equipe

7.1. Pontos fortes

- A divisão por área permaneceu estável e o histórico preservou a autoria dos três integrantes.

- As tarefas permaneceram identificadas por Jira, mesmo quando atividades relacionadas compartilharam a mesma integração.
- As evidências reproduzíveis cobriram API, segurança, carga e clone limpo. Problemas encontrados pelo QA produziram correções no backend e no frontend.

7.2. Pontos fracos

- A cadência não foi uniforme e houve quedas concentradas no burndown.
- A capacidade de duas buscas simultâneas protege a aplicação, mas é baixa para picos de acesso.
- A documentação final ficou concentrada no encerramento, exigindo conferência simultânea do Jira, histórico Git e evidências.
- A importação pública e os cenários positivos do frontend não ficaram registrados com evidências finais reproduzíveis.

7.3. Dificuldades e impedimentos

Período	Dificuldade ou impedimento	Ação e resultado
12/09	Estrutura inicial diferente da convenção do enunciado.	Backend, frontend, banco e composição foram mantidos diretamente na raiz do repositório.
13/09	Três integrantes no mesmo computador precisavam preservar identidades distintas.	Clones e identidades separadas mantiveram a autoria dos commits.
15/09	A validação das imagens dependia do Docker em execução.	O ambiente passou a ser iniciado antes das verificações.
17–25/09	Interrupções e dias sem integração acumularam entregas.	O trabalho foi retomado em lotes, com commits atômicos e chaves Jira.
04/10	A carga confirmou baixa capacidade e custo da busca ampla por similaridade.	Limite, timeout e índice fizeram o excesso falhar de maneira controlada.
10/10	O relatório inicial não aplicava o pacote oficial da SBC nem priorizava os itens avaliados.	O documento foi recomposto com o template oficial e reorganizado pelo enunciado.

8. Preparação da entrega

O repositório mantém `backend/`, `frontend/`, `db/`, `docs/` e `docker-compose.yml` diretamente na raiz. Para atender à estrutura do arquivo final sem duplicar diretórios durante o desenvolvimento, o ZIP de entrega deve envolver essa raiz em uma única pasta externa chamada `projeto/`. Os arquivos dos conjuntos públicos e o banco preenchido não devem ser incluídos.

Depois da extração, o avaliador deverá entrar em `projeto/` e executar `docker compose up --build`. O procedimento inicia os serviços. Como registrado nas limitações, a realização de buscas com resultados depende da preparação e importação dos dados públicos.

9. Conclusão

O projeto produziu uma aplicação web modularizada em PostgreSQL, API Flask com Gunicorn e frontend React. Foram implementadas buscas por nome exato, similaridade, cargo, UF, órgão e combinações, além de paginação, respostas JSON padronizadas, validação, limite de concorrência, timeout e tratamento dos erros previstos.

A Sprint 1 encerrou com 56 de 56 pontos e a Sprint 2 com 59 de 59, totalizando 115 Story Points do backlog registrados como concluídos. Esse fechamento do planejamento não significa atendimento integral aos requisitos classificados como parciais na Tabela 6. As tarefas e os commits dos três integrantes estão documentados por dia, juntamente com o burndown e a análise crítica.

As evidências de 04/10 confirmam o backend, a proteção contra injeção e o controle de sobrecarga naquela versão; os 12 testes automatizados passaram novamente na validação de clone limpo de 09/10. As baterias completas não foram repetidas após a correção de 06/10. A avaliação final deve considerar também a ausência de carga automática dos dados públicos, de contagem final importada e de evidência positiva completa do frontend. O aplicativo desktop permaneceu fora do escopo obrigatório.

10. Disponibilidade do código e evidências

O código-fonte e o histórico Git estão disponíveis em `https://github.com/marcoscfreitas/spi-ii-3bim`. As evidências usadas neste relatório incluem:

- `docs/analise-datasets.md`, `docs/mapeamento-campos.md` e `docs/modelo-banco.md`;
- `backend/README.md`, `docs/plano-testes.md` e `docs/casos-sql-injection.md`;
- a validação preliminar da composição, de 18/09, em `docs/validacao-compose.md`; a validação final de clone limpo, de 09/10, está resumida neste relatório e no histórico da PROJ-50, sem log independente;
- o acompanhamento em `docs/acompanhamento-sprint-1.md` e o gráfico em `docs/burndown.svg`;
- os registros JSON em `docs/evidencias/`, produzidos em 04/10/2026;
- o histórico integrado da branch `dev`, usado para conferir autoria, datas e mensagens.

Referências

Docker (2026). Docker compose quickstart. `https://docs.docker.com/compose/gettingstarted/`. Acesso em: 9 out. 2026.

Ministério da Gestão e da Inovação em Serviços Públicos (2018). Dicionário de dados de aposentados. `https://repositorio.dados.gov.br/segrt/1.20180829-DadosRecursos-Aposentados.pdf`. Acesso em: 9 out. 2026.

Ministério da Gestão e da Inovação em Serviços Públicos (2019). Dicionário de dados de carreiras e cargos. https://repositorio.dados.gov.br/segrt/2.20190709-DadosRecursos-Carreiras.pdf. Acesso em: 9 out. 2026.

Portal Brasileiro de Dados Abertos (2026a). Gestão de pessoas do executivo federal – aposentados. https://dados.gov.br/dados/conjuntos-dados/gestao-de-pessoas-executivo-federal--aposentados. Acesso em: 9 out. 2026.

Portal Brasileiro de Dados Abertos (2026b). Gestão de pessoas do executivo federal – carreiras/cargos. https://dados.gov.br/dados/conjuntos-dados/gestao-de-pessoas-executivo-federal---carreiras--cargos. Acesso em: 9 out. 2026.

Sistemas para Internet II (2026). Trabalho do terceiro bimestre: Desenvolvimento de uma aplicação full stack. Enunciado da atividade, Universidade Federal do Rio Grande.